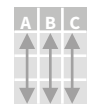


Data transformation with dplyr : : CHEATSHEET



dplyr functions work with pipes and expect **tidy data**. In tidy data:



&



pipes

Each **variable** is in its own **column**

Each **observation**, or **case**, is in its own **row**

x > **f(y)** becomes **f(x, y)**

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function



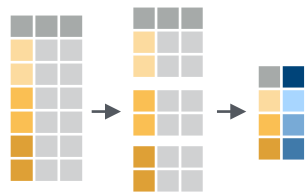
summarize(.data, ...)
Compute table of summaries.
mtcars > summarize(avg = mean(mpg))



count(.data, ..., wt = NULL, sort = FALSE, name = NULL) Count number of rows in each group defined by the variables in ... Also **tally()**, **add_count()**, **add_tally()**.
mtcars > count(cyl)

Group Cases

Use **group_by(.data, ..., .add = FALSE, .drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



mtcars >
group_by(cyl) >
summarize(avg = mean(mpg))

Alternate grouping syntax with **.by** as an argument:

mtcars >
summarize(
avg = mean(mpg), .by = cyl
)

Use **rowwise(.data, ...)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidy cheat sheet for list-column workflow.



starwars >
rowwise() >
mutate(film_count = length(films))

ungroup(x, ...) Returns ungrouped copy of table.

g_mtcars <- mtcars > group_by(cyl)
ungroup(g_mtcars)

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.



filter(.data, ..., .preserve = FALSE) Extract rows that meet logical criteria.
mtcars > filter(mpg > 20)

filter_out(.data, ..., .preserve = FALSE): Drop rows that meet logical criteria (the complement to `filter()`). Missing values in the dropped condition are kept by default, so you don't need to guard against NA explicitly.
mtcars > filter_out(mpg > 20)

Use **when_any()** and **when_all()** inside **filter()** to combine multiple conditions.
mtcars > filter(when_any(mpg > 30, hp > 300))



distinct(.data, ..., .keep_all = FALSE) Remove rows with duplicate values.
mtcars > distinct(gear)



slice(.data, ..., .preserve = FALSE) Select rows by position.
mtcars > slice(10:15)



slice_sample(.data, ..., n, prop, weight_by = NULL, replace = FALSE) Randomly select rows. Use n to select a number of rows and prop to select a fraction of rows.
mtcars > slice_sample(n = 5, replace = TRUE)



slice_min(.data, order_by, ..., n, prop, with_ties = TRUE) and **slice_max()** Select rows with the lowest and highest values.
mtcars > slice_min(mpg, prop = 0.25)

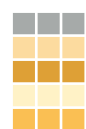
slice_head(.data, ..., n, prop) and **slice_tail()** Select the first or last rows.
mtcars > slice_head(n = 5)

Logical and boolean operators to use with filter()

==	<	<=	is.na()	%in%		xor()
!=	>	>=	!is.na()	!	&	

See **?base::Logic** and **?Comparison** for help.

ARRANGE CASES



arrange(.data, ..., .by_group = FALSE) Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
mtcars > arrange(mpg)
mtcars > arrange(desc(mpg))

ADD CASES



add_row(.data, ..., .before = NULL, .after = NULL) Add one or more rows to a table.
cars > add_row(speed = 1, dist = 1)

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.



pull(.data, var = -1, name = NULL, ...) Extract column values as a vector, by name or index.
mtcars > pull(wt)



select(.data, ...) Extract columns as a table.
mtcars > select(mpg, wt)



relocate(.data, ..., .before = NULL, .after = NULL) Move columns to new position.
mtcars > relocate(mpg, cyl, .after = last_col())

Use these helpers with select() and across()

e.g. mtcars > select(mpg:cyl)

contains(match)	num_range(prefix, range)	; e.g., mpg:cyl
ends_with(match)	all_of(x)/any_of(x, ..., vars)	! e.g., !gear
starts_with(match)	matches(match)	everything()

MANIPULATE MULTIPLE VARIABLES AT ONCE

df <- tibble(x_1 = c(1, 2), x_2 = c(3, 4), y = c(4, 5))



across(.cols, .funs, ..., .names = NULL) Summarize or mutate multiple columns in the same way.
df > summarize(across(everything(), mean))



c_across(.cols) Compute across columns in row-wise data.
df >
rowwise() >
mutate(x_total = sum(c_across(1:2)))

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

vectorized function



mutate(.data, ..., .keep = "all", .before = NULL, .after = NULL) Compute new column(s). Also **add_column()**.
mtcars > mutate(gpm = 1 / mpg)
mtcars > mutate(gpm = 1 / mpg, .keep = "none")



rename(.data, ...) Rename columns. Use **rename_with()** to rename with a function.
mtcars > rename(miles_per_gallon = mpg)

Vectorized Functions

TO USE WITH MUTATE ()

mutate() applies vectorized functions to columns to create new columns. Vectorized functions take vectors as input and return vectors of the same length as output.

vectorized function

OFFSET

`dplyr::lag()` - offset elements by 1
`dplyr::lead()` - offset elements by -1

CUMULATIVE AGGREGATE

`dplyr::cumall()` - cumulative all()
`dplyr::cumany()` - cumulative any()
`dplyr::cummax()` - cumulative max()
`dplyr::cummean()` - cumulative mean()
`dplyr::cummin()` - cumulative min()
`dplyr::cumprod()` - cumulative prod()
`dplyr::cumsum()` - cumulative sum()

RANKING

`dplyr::cume_dist()` - proportion of all values <=
`dplyr::dense_rank()` - rank w ties = min, no gaps
`dplyr::min_rank()` - rank with ties = min
`dplyr::ntile()` - bins into n bins
`dplyr::percent_rank()` - min_rank scaled to [0,1]
`dplyr::row_number()` - rank with ties = "first"

MATH

`+`, `-`, `*`, `/`, `^`, `%/%`, `%%` - arithmetic ops
`log()`, `log2()`, `log10()` - logs
`<`, `<=`, `>`, `>=`, `!=`, `==` - logical comparisons
`dplyr::between()` - `x >= left & x <= right`
`dplyr::near()` - safe `==` for floating point numbers

MISCELLANEOUS

`dplyr::if_else()` - element-wise if() + else()
`dplyr::na_if()` - replace specific values with NA
`dplyr::case_when()` - multi-case if_else(). Use `.unmatched = "error"` to error on any unmatched cases instead of silently returning NA.

```
starwars |>
  mutate(type = case_when(
    height > 200 | mass > 200 ~ "large",
    species == "Droid" ~ "robot",
    .default = "other"
  ))
dplyr::replace_when() - replace values based on a logical condition, like a targeted case_when().
```

```
starwars |>
  mutate(name = replace_when(
    name, height > 200 ~ "Tall")
  )
```

`dplyr::coalesce()` - first non-NA values by element across a set of vectors
`dplyr::recode_values()` - recode values into a new column, using either a formula interface (like `case_when()`) or from / to lookup vectors.

```
starwars |>
  mutate(name = recode_values(
    species, "Droid" ~ "robot", default = "other"
  ))
```

`dplyr::replace_values()` - replace specific values within an existing column, similar to `coalesce()`, `na_if()`, or `tidyr::replace_na()`.

```
starwars |>
  mutate(name = replace_values(
    species, NA ~ "Unknown"
  ))
```

`dplyr::pmax()` - element-wise max()
`dplyr::pmin()` - element-wise min()

Summary Functions

TO USE WITH SUMMARIZE ()

summarize() applies summary functions to columns to create a new table. Summary functions take vectors as input and return single values as output.

summary function

COUNT

`dplyr::n()` - number of values/rows
`dplyr::n_distinct()` - # of uniques
`sum(!is.na())` - # of non-NAs

POSITION

`mean()` - mean, also `mean(!is.na())`
`median()` - median

LOGICAL

`mean()` - proportion of TRUEs
`sum()` - # of TRUEs

ORDER

`dplyr::first()` - first value
`dplyr::last()` - last value
`dplyr::nth()` - value in nth location of vector

RANK

`quantile()` - nth quantile
`min()` - minimum value
`max()` - maximum value

SPREAD

`IQR()` - Inter-Quartile Range
`mad()` - median absolute deviation
`sd()` - standard deviation
`var()` - variance

Row Names

Tidy data does not use rownames, which store a variable outside of the columns. To work with the rownames, first move them into a column.

`tibble::rownames_to_column()`
Move row names into col.
`a <- mtcars |>`
`rownames_to_column(var = "C")`

`tibble::column_to_rownames()`
Move col into row names.
`a |> column_to_rownames(var = "C")`

Also `tibble::has_rownames()` and `tibble::remove_rownames()`.

Combine Tables

COMBINE VARIABLES

`x` + `y` =

A	B	C	E	F	G
a	t	1	a	t	3
b	u	2	b	u	2
c	v	3	c	v	3

bind_cols(..., .name_repair) Returns tables placed side by side as a single table. Column lengths must be equal. Columns will NOT be matched by id (to do that look at Relational Data below), so be sure to check that both tables are ordered the way you want before binding.

RELATIONAL DATA

`x` + `y` =

A	B	C	D
a	t	1	a
b	u	2	b
c	v	3	d

Use a "**Mutating Join**" to join one table to columns from another, matching values with the rows that they correspond to. Each join retains a different combination of values from the tables.

`left_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na")` Join matching values from y to x.

`right_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na")` Join matching values from x to y.

`inner_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na")` Join data. Retain only rows with matches.

`full_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na")` Join data. Retain all values, all rows.

COLUMN MATCHING FOR JOINS

Use **by = c("col1", "col2", ...)** to specify one or more common columns to match on.
`left_join(x, y, by = "A")`

Use a named vector, **by = c("col1" = "col2")**, to match on columns that have different names in each table.
`left_join(x, y, by = c("C" = "D"))`

Use **suffix** to specify the suffix to give to unmatched columns that have the same name in both tables.
`left_join(x, y, by = c("C" = "D"), suffix = c("1", "2"))`

COMBINE CASES

`x` + `y` =

A	B	C	D
x	a	t	1
y	c	v	3
y	d	w	4

bind_rows(..., .id = NULL) Returns tables one on top of the other as a single table. Set `.id` to a column name to add a column of the original table names (as pictured).

Use a "**Filtering Join**" to filter one table against the rows of another.

`semi_join(x, y, by = NULL, copy = FALSE, ..., na_matches = "na")` Return rows of x that have a match in y. Use to see what will be included in a join.

`anti_join(x, y, by = NULL, copy = FALSE, ..., na_matches = "na")` Return rows of x that do not have a match in y. Use to see what will not be included in a join.

Use a "**Nest Join**" to inner join one table to another into a nested data frame.

`nest_join(x, y, by = NULL, copy = FALSE, keep = FALSE, name = NULL, ...)` Join data, nesting matches from y in a single new data frame column.

SET OPERATIONS

`intersect(x, y, ...)`
Rows that appear in both x and y.

`setdiff(x, y, ...)`
Rows that appear in x but not y.

`union(x, y, ...)`
Rows that appear in x or y, duplicates removed). **union_all()** retains duplicates.

Use **setequal()** to test whether two data sets contain the exact same rows (in any order).

